

Constraint-Based Problem Solving

a) PEAS Description of the Exam-Scheduling Agent

Component	Description
Performance Measure	Produce a complete, conflict-free timetable for all five courses using the three available slots. Ensure that every conflicting pair is assigned to different slots and that all courses are scheduled.
Environment	The university examination-scheduling problem: five courses (A, B, C, D, E), three exam slots (Slot1, Slot2, Slot3), and the specified student-enrolment conflicts.
Actuators	The scheduling mechanism that assigns a course to a slot, revises an assignment when necessary, and outputs the final timetable.
Sensors	The course list, available slots, conflict information, current partial timetable, and remaining legal domain values for unassigned courses.

b) Most Suitable Type of Intelligent Agent

A goal-based agent is the most suitable choice. Its goal is to find a timetable in which every course is assigned to one of the three slots and no conflicting pair shares a slot. It evaluates possible assignments against this goal, explores alternatives when a conflict occurs, and stops when it finds a valid timetable.

A simple reflex agent would be limited to immediate condition–action rules and may not handle interacting constraints well. A model-based agent could maintain the current timetable and constraints, but the central requirement is to search for a state that satisfies a defined goal. A utility-based agent would be more useful if several valid timetables had to be ranked by preferences such as student convenience or room usage; no such preferences are specified here.

c) CSP Formulation and Constraint Graph

A Constraint Satisfaction Problem (CSP) consists of variables, domains, and constraints.

Variables

A, B, C, D, and E represent the exam slots assigned to Artificial Intelligence, Database Systems, Computer Networks, Operating Systems, and Software Engineering, respectively.

Domains

For every variable: {Slot1, Slot2, Slot3}.

Constraints

Each conflicting pair must have different slot values:

- $A \neq B$
- $A \neq C$
- $B \neq C$
- $B \neq D$
- $C \neq D$
- $C \neq E$
- $D \neq E$

Constraint Graph

Vertices: {A, B, C, D, E}

Edges: {(A,B), (A,C), (B,C), (B,D), (C,D), (C,E), (D,E)}

An edge connects two courses that cannot be scheduled in the same slot. A valid solution assigns a slot to every vertex so that the endpoints of every edge have different values.

Graph sketch (edges shown as pairs):

A — B; A — C; B — C; B — D; C — D; C — E; D — E

d) Plain Backtracking Search

Variable order: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$. For each variable, try Slot1 $\rightarrow$ Slot2 $\rightarrow$ Slot3. Reject a value only when it conflicts with an already assigned neighbouring course.

Step	Variable / attempt	Result	Explanation / current assignments
1	A = Slot1	Accept	No earlier assignments; A = Slot1.
2	B = Slot1	Reject	A and B conflict, so they cannot share Slot1.
3	B = Slot2	Accept	B = Slot2; it differs

			from A.
4	C = Slot1	Reject	C conflicts with A, which is in Slot1.
5	C = Slot2	Reject	C conflicts with B, which is in Slot2.
6	C = Slot3	Accept	C = Slot3; it differs from A and B.
7	D = Slot1	Accept	D differs from B (Slot2) and C (Slot3).
8	E = Slot1	Reject	E conflicts with D, which is in Slot1.
9	E = Slot2	Accept	E differs from C (Slot3) and D (Slot1). All variables are assigned.

Final valid timetable:

Course	Slot	Course name
A	Slot1	Artificial Intelligence
B	Slot2	Database Systems
C	Slot3	Computer Networks
D	Slot1	Operating Systems
E	Slot2	Software Engineering

The search encounters two rejected value attempts (B = Slot1 and E = Slot1). Under this specified variable/value order, neither rejection requires undoing an earlier variable assignment, so there are zero variable-level backtracks. The search then reaches a complete solution.

e) Backtracking Search with Forward Checking

Forward checking removes a selected slot from the domains of unassigned neighbouring variables immediately after an assignment. If any unassigned variable has an empty domain, the search must backtrack. Domain values removed by forward checking are not attempted later.

Step	Assignment / action	Domains after propagation (unassigned variables)	Outcome
Initial	No assignment	B,C,D,E: {Slot1, Slot2, Slot3}	Start with full domains.
1	Try A = Slot1	B: {Slot2, Slot3}; C: {Slot2, Slot3}; D: {1,2,3}; E: {1,2,3}	Remove Slot1 from B and C because A conflicts with both.
2	Try B = Slot1, then B = Slot2	After B = Slot2: C: {Slot3}; D: {Slot1, Slot3}; E: {Slot1, Slot2, Slot3}	B = Slot1 is rejected because A = Slot1. B = Slot2 removes Slot2 from C and D.
3	Assign C = Slot3	D: {Slot1}; E: {Slot1,	C has only Slot3 left.

		Slot2}	Remove Slot3 from D and E.
4	Assign D = Slot1	E: {Slot2}	D conflicts with E, so remove Slot1 from E. E is forced to Slot2.
5	Assign E = Slot2	No unassigned variables	Complete conflict-free assignment.

Final timetable: A = Slot1, B = Slot2, C = Slot3, D = Slot1, E = Slot2.

Note: In the domain table, Slot1/Slot2/Slot3 may be abbreviated as 1/2/3. Domains are shown after applying the constraints from assigned courses.

f) Comparison of the Two Methods

Measure	Plain Backtracking	Backtracking + Forward Checking
Assignment/value attempts	9 value attempts: A(1), B(2), C(3), D(1), E(2).	6 value attempts: A(1), B(2), C(1), D(1), E(1). Values removed from domains are not attempted.
Rejected attempts	2: B = Slot1 and E = Slot1.	1 direct rejected attempt: B = Slot1. Other impossible values are pruned before they are attempted.
Backtracks	0 variable-level backtracks for this ordering; two individual values are rejected.	0 variable-level backtracks; forward checking detects and removes inconsistent values early.
Efficiency	More assignments are tried because conflicts with future variables are not propagated in advance.	Fewer assignment attempts because the domains of unassigned neighbours are reduced after each assignment.

Why forward checking reduces search effort: After A is assigned Slot1, Slot1 is removed from B and C. After B is assigned Slot2, C is forced to Slot3. After C is assigned Slot3, D is forced to Slot1; then E is forced to Slot2. The algorithm therefore avoids trying values already known to be inconsistent with current assignments.

PROGRAMS 1:

ai 3.py - C:/Users/sridh/AppData/Local/Programs/Python/Python314/ai 3.py (3.14.7)

File Edit Format Run Options Window Help

```
rejected = 0

def is_valid(course, slot):
    for neighbour in constraints[course]:
        if neighbour in assignment:
            if assignment[neighbour] == slot:
                return False
    return True

def backtracking():
    global attempts, rejected

    # All courses have been scheduled
    if len(assignment) == len(courses):
        return True

    # Select the next unassigned course
    course = next(c for c in courses if c not in assignment)

    # Try each slot
    for slot in slots:
        attempts += 1

        if is_valid(course, slot):
            assignment[course] = slot

            if backtracking():
                return True

            # Undo assignment if it leads to failure
            del assignment[course]
        else:
            rejected += 1

    return False

if backtracking():
    print("Valid Exam Timetable")
    for course in courses:
        print(course, ":", assignment[course])
else:
    print("No solution exists")

print("Assignment attempts:", attempts)
print("Rejected attempts:", rejected)
```

OUTPUT:

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/sridh/AppData/Local/Programs/Python/Python314/ai 3.py ====
Valid Exam Timetable
A : Slot1
B : Slot2
C : Slot3
D : Slot1
E : Slot2
Assignment attempts: 9
Rejected attempts: 4
>>>
```

PROGRAM 2:

 *untitled (3.14.7)*

File Edit Format Run Options Window Help

```
# Exam Scheduling using Forward Checking

courses = ['A', 'B', 'C', 'D', 'E']
slots = ['Slot1', 'Slot2', 'Slot3']

constraints = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D', 'E'],
    'D': ['B', 'C', 'E'],
    'E': ['C', 'D']
}

assignment = {}
attempts = 0
rejected = 0

def forward_check(course, slot, domains):
    new_domains = {
        c: values.copy()
        for c, values in domains.items()
    }

    # Remove the selected slot from neighbours
    for neighbour in constraints[course]:
        if neighbour not in assignment and neighbour in new_domains:
            if slot in new_domains[neighbour]:
                new_domains[neighbour].remove(slot)

    # Empty domain means failure
    if not new_domains[neighbour]:
        return None

    return new_domains

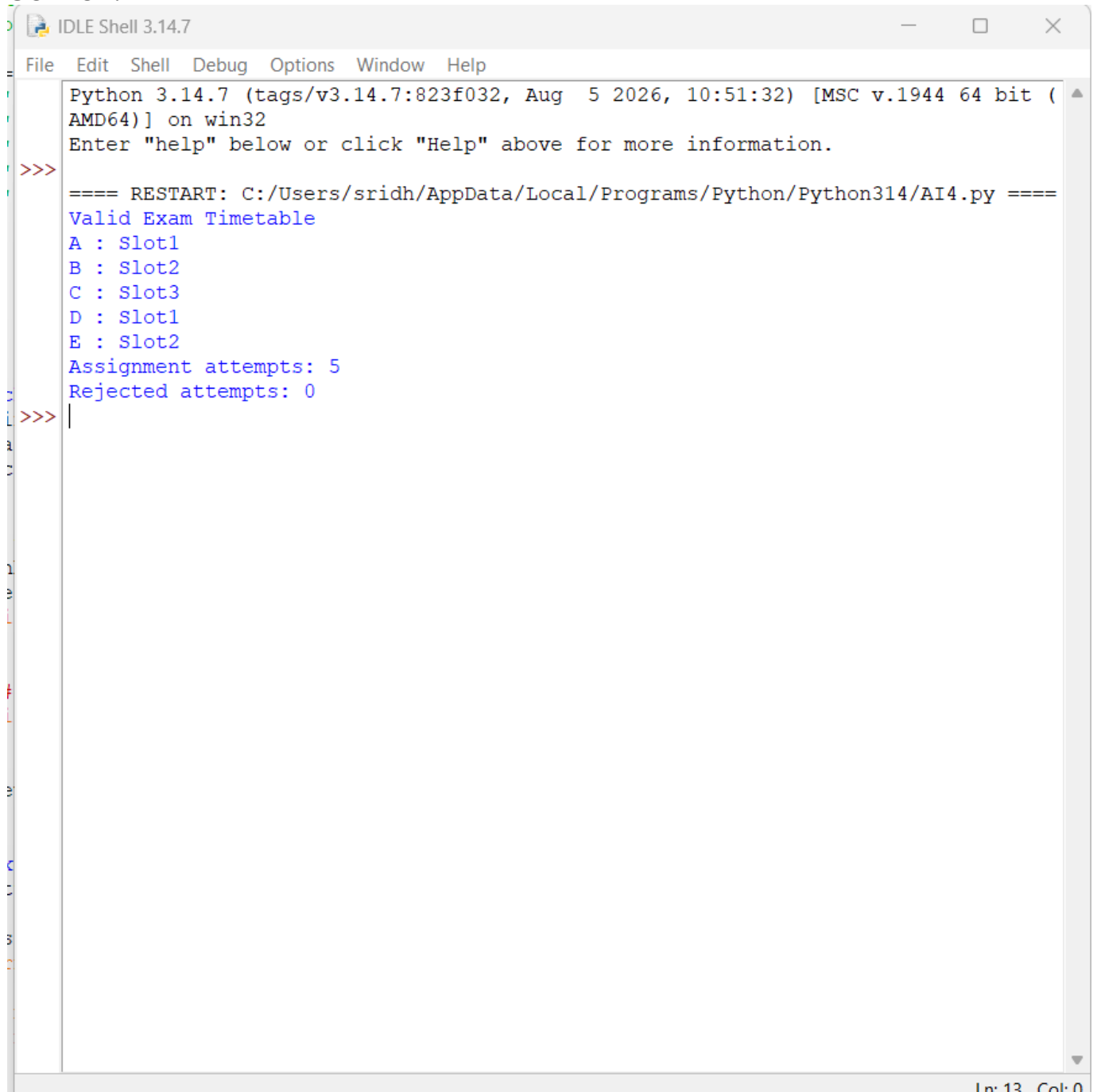
def backtracking(domains):
    global attempts, rejected

    if len(assignment) == len(courses):
        return True

    # Select next unassigned course
    course = next(c for c in courses if c not in assignment)

    for slot in domains[course]:
        attempts += 1
```

OUTPUT:



```
IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/sridh/AppData/Local/Programs/Python/Python314/AI4.py ====
Valid Exam Timetable
A : Slot1
B : Slot2
C : Slot3
D : Slot1
E : Slot2
Assignment attempts: 5
Rejected attempts: 0
>>> |
Ln 13, Col 0
```

CONCLUSION:

The CSP has a valid solution: Artificial Intelligence—Slot1, Database Systems—Slot2, Computer Networks—Slot3, Operating Systems—Slot1, and Software Engineering—Slot2. Every listed conflicting pair receives different slots. Forward checking reaches the same solution with fewer assignment attempts by pruning domains early.